

✅ 0410 실험 계획 및 결과

- 비선형 함수 추가 (ex. cos)
 - $f(x) = a \cdot x + b \cdot \cos(c + d \cdot x) + e$ 형태 수식 생성
 - PySR로 수식 도출
 - `maxsize=15` 으로 설정해서 수식 생성 완료
 - 다른 stroke 데이터에 적용
 - RMSE, R^2 측정 완료
 - 각 stroke에 따른 파라미터 a~e 시각화
 - 파라미터 예측 모델로 전환
 - 수식을 매번 생성하지 않고, 각 계수를 예측하는 모델을 구축
- 결론: 수식이 너무 단순하다
- `maxsize=15` 는 부족하고, `20~25` 정도로 늘려야 될 것으로 보임

아래는 금일 시도한 실험 전개임

🔍 Polynomial 다항식에 sin term 추가

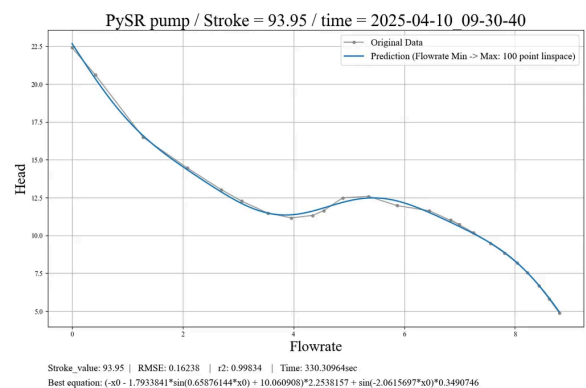
polynomial 다항식에 sin term 추가

sin vs cos term 실험 비교

TODO(PC24-11, 2025-04-10 09:20)

1 Polynomial + sin

```
niterations=700
populations=70
population_size=70
binary_operators=["+", "-", "*", "^"]
unary_operators=["sin"]
constraints={"^":(-1, 0), "sin":(-1,2)}
nested_constraints={"^":{"^": 0}, "sin":
{"sin":0}}
maxsize=20
```

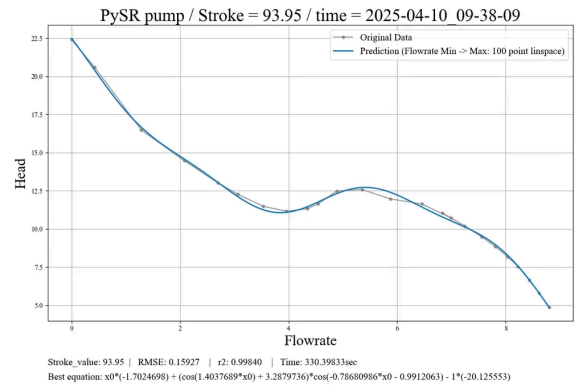


✅ 최종 완전 전개된 수식:

$$f(x) = -2.2538157x - 4.042454 \cdot \sin(0.65876144x) - 0.3490746 \cdot \sin(2.0615697x) + 22.684149$$

2 Polynomial + cos

```
niterations=700
populations=70
population_size=70
binary_operators=["+", "-", "*", "^"]
unary_operators=["cos"]
constraints={"^":(-1, 0), "cos":(-1,2)}
nested_constraints={"^":{"^": 0}, "cos":
{"cos":0}}
maxsize=2
```



✅ 최종 전개 결과:

$$f(x) = -1.7024698x + \cos(1.4037689x) \cdot \cos(0.78680986x + 0.9912063) + 3.2879736 \cdot \cos(0.78680986x + 0.9912063) + 20.125553$$

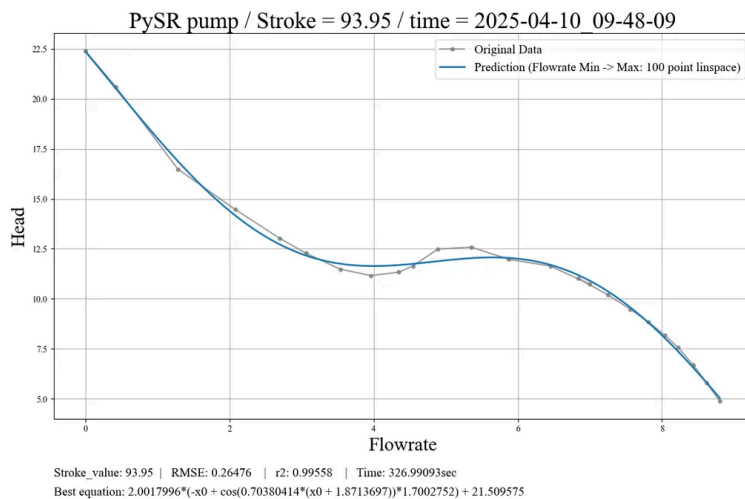
📌 결론:

📄 PySR에서 sin과 cos 차이

sin과 cos 은 2개의 차이가 없으므로 cos(x)만으로도 충분하다고 판단이 됨

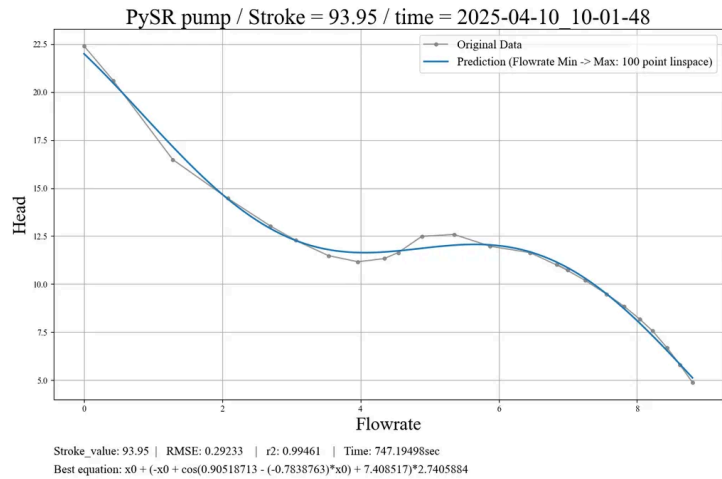
🔍 수식이 길어서 maxsize를 20에서 15로 조정하여 다시 실험

maxsize =15로 두고 다시 실험



🔍 maxsize를 줄여서 수식의 길이는 줄였지만 근사가 잘 안되어서 탐색 공간을 늘려서 다시 실험

iterations=700 → 1000
 populations=70 → 100
 population_size=70 → 100



현재 수식 치환 →

✅ 최종 정리된 전개 수식:

$$f(x) = -1.7405884x + 2.7405884 \cdot \cos(0.90518713 + 0.7838763x) + 20.303781$$

ID 치환 방식:

기호	의미
a	x 의 선형 계수
b	cos 앞의 계수
c	cos 안의 상수항
d	cos 안의 x 계수
e	상수항 (bias term)

👉 치환된 수식:

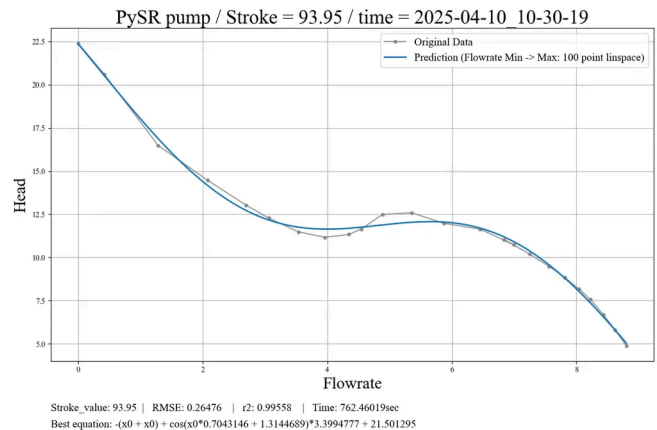
$$f(x) = a \cdot x + b \cdot \cos(c + d \cdot x) + e$$

📌 치환 값:

$$\begin{aligned}
 a &= -1.7405884 \\
 b &= 2.7405884 \\
 c &= 0.90518713 \\
 d &= 0.7838763 \\
 e &= 20.303781
 \end{aligned}$$

🔍 탐색 범위를 다르게 해서 다시 실험

iterations=700 → 1000 → 1000
populations=70 → 100 → 50
population_size=70 → 100 → 300



✅ 최종 전개된 수식:

$$f(x) = -2x + 3.3994777 \cdot \cos(0.7043146x + 1.3144689) + 21.501295$$

🎯 원래 전개된 수식:

$$f(x) = \underbrace{-2}_a x + \underbrace{3.3994777}_b \cdot \cos\left(\underbrace{0.7043146}_d x + \underbrace{1.3144689}_c\right) + \underbrace{21.501295}_e$$

✅ 치환된 일반식:

$$f(x) = a \cdot x + b \cdot \cos(c + d \cdot x) + e$$

📋 계수 값:

기호	값
a	-2
b	3.3994777
c	1.3144689
d	0.7043146
e	21.501295

📌 결론:

maxsize 설정이 모델 성능에 크게 영향을 미치는 것을 확인함

이제 동일한 PySR 조건에서, 7개의 stroke 데이터를 각각 실험하여 수식 구조 및 계수의 변화 양상을 분석예정

● 추후 maxsize 15로 최적화 필요 ●

현재는 다음 단계로 가기위하여 15로 설정하였지만 최적화가 안되면 20으로 해야될 수도 있음 아래는 maxsize를 15로 설정하여 실험 진행한 결과 임

7개의 다른 조건(stroke)으로 PySR 실험

PySR 설정, random_state고정

공통 설정:

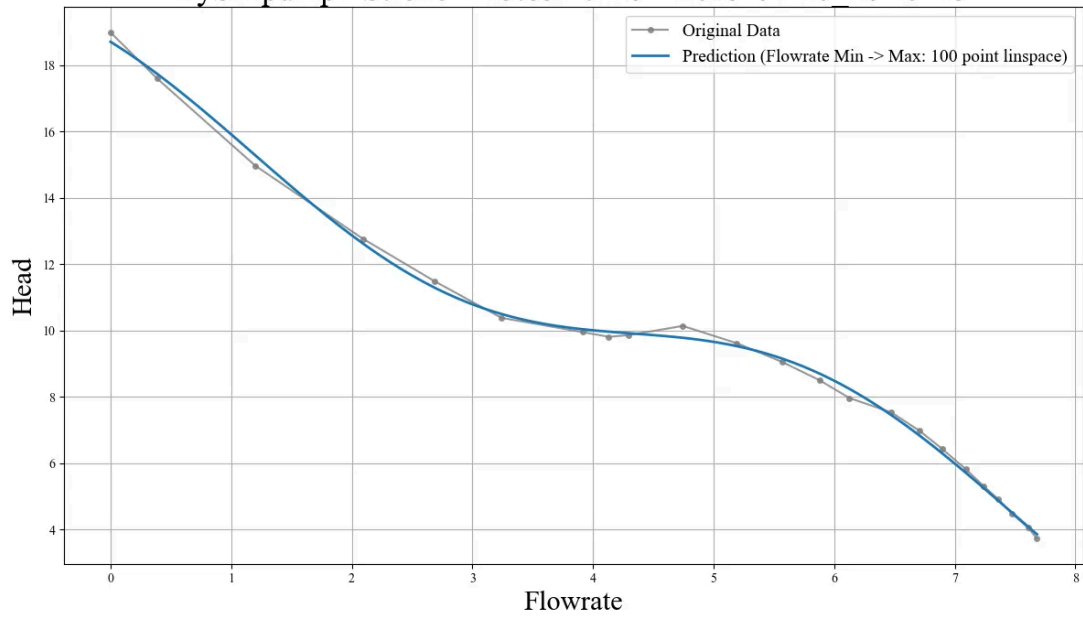
```
niterations=1000
populations=50
population_size=300
binary_operators=["+", "-", "*", "^"],
unary_operators=["cos"],
constraints={"^": (-1, 0), "cos": (-1, 2)}
nested_constraints = {"cos" : {"cos": 0}, "^": {"^": 0}}
maxsize=15, parsimony=0, # 수식 복잡도에 대한 벌점 (페널티)
random_state=42
```

실험 결과에서 관측해야 하는 것들

- 1) 수식 구조 변화
- 2) 계수 변화(a,b,c,d,e)
- 3) 예측 성능
- 4) 수식의 복잡도
- 5) 공통된 패턴 찾기
- 6) 그래프 비교

 Data 1: stroke=15.85

PySR pump / Stroke = 15.85 / time = 2025-04-10 10-49-45



Stroke_value: 15.85 | RMSE: 0.16761 | r2: 0.99822 | Time: 752.20772sec
Best equation: $17.361654 - 1.7189162 \cdot (x0 - 0.84364223 \cdot \cos(x0 + 0.3792143))$

✅ 최종 전개된 수식:

$$f(x) = -1.7189162 \cdot x + 1.450246 \cdot \cos(x + 0.3792143) + 17.361654$$

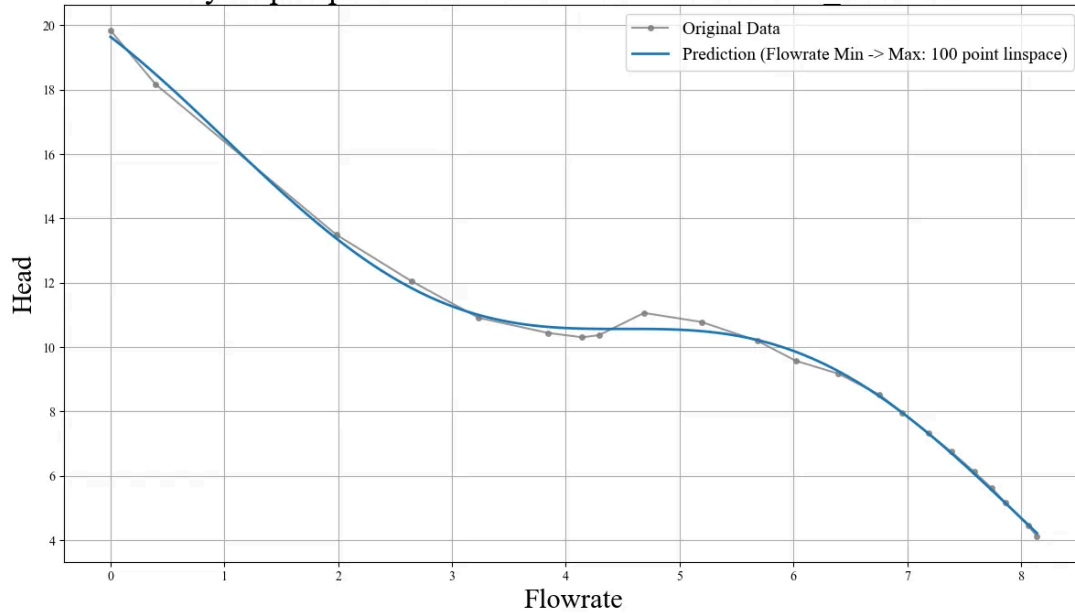
📋 계수 정리 (a, b, c, d, e):

$$f(x) = a \cdot x + b \cdot \cos(c + d \cdot x) + e$$

항목	값
a	-1.7189162
b	1.450246
c	0.3792143
d	1 (= x 그대로 들어감)
e	17.361654

🔍 Data 2: stroke=45.8

PySR pump / Stroke = 45.8 / time = 2025-04-10 11-10-17

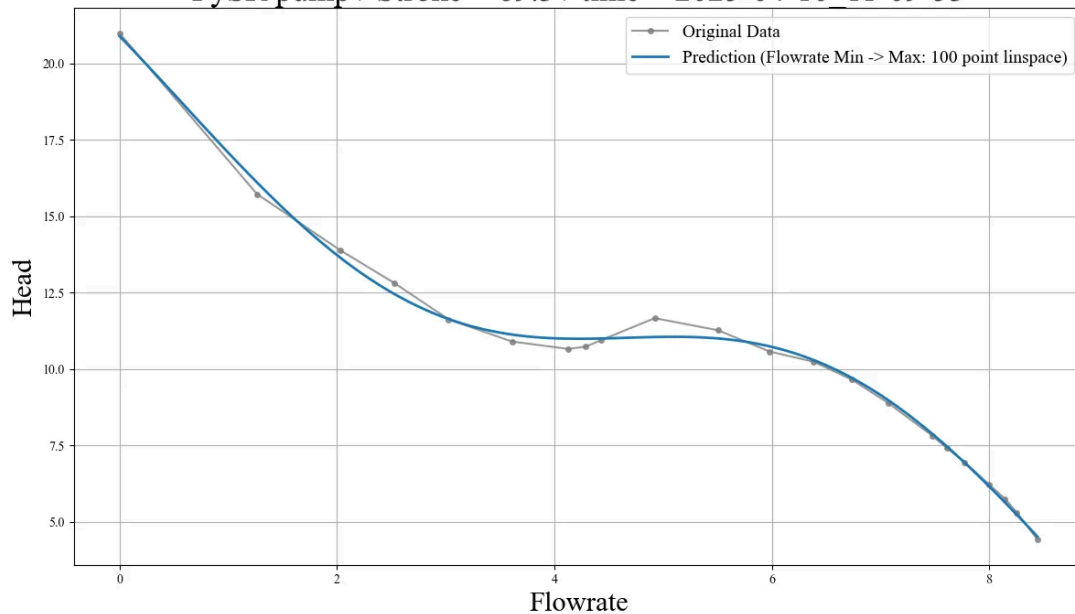


✓ 최종 전개 수식:

$$f(x) = -1.684188x + 1.879294 \cdot \cos(0.8985187x + 0.66195387) + 18.144154$$

🔍 Data 3: stroke=69.3

PySR pump / Stroke = 69.3 / time = 2025-04-10 11-09-53

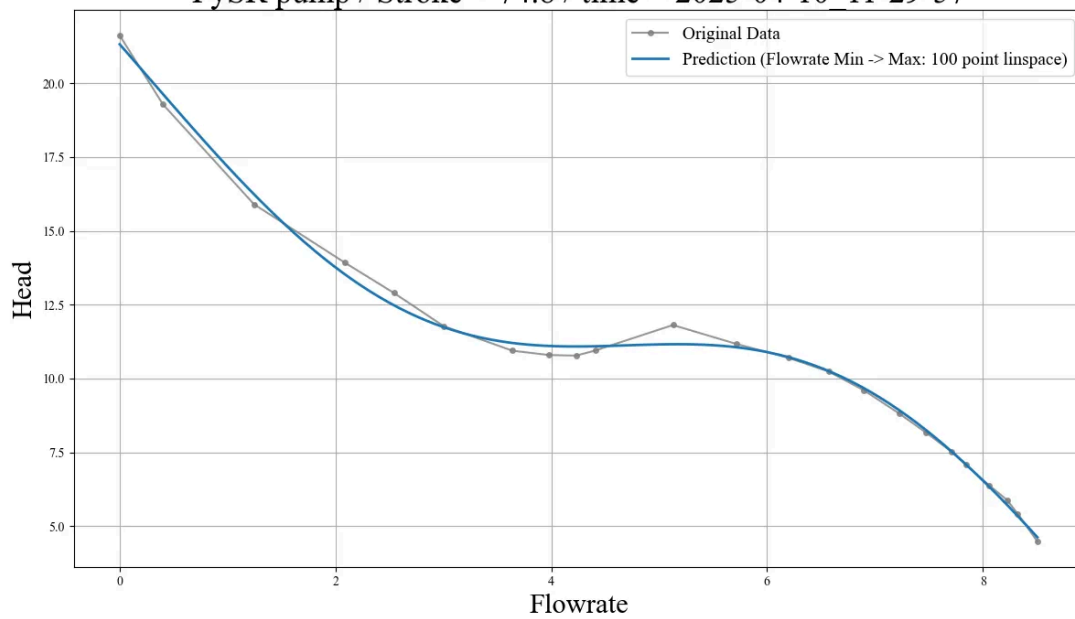


✓ 정리된 최종 수식:

$$f(x) = -1.875963x + 2.581567 \cdot \cos(0.7695822x + 1.1242205) + 19.765572$$

Data 4: stroke=74.8

PySR pump / Stroke = 74.8 / time = 2025-04-10_11-29-57



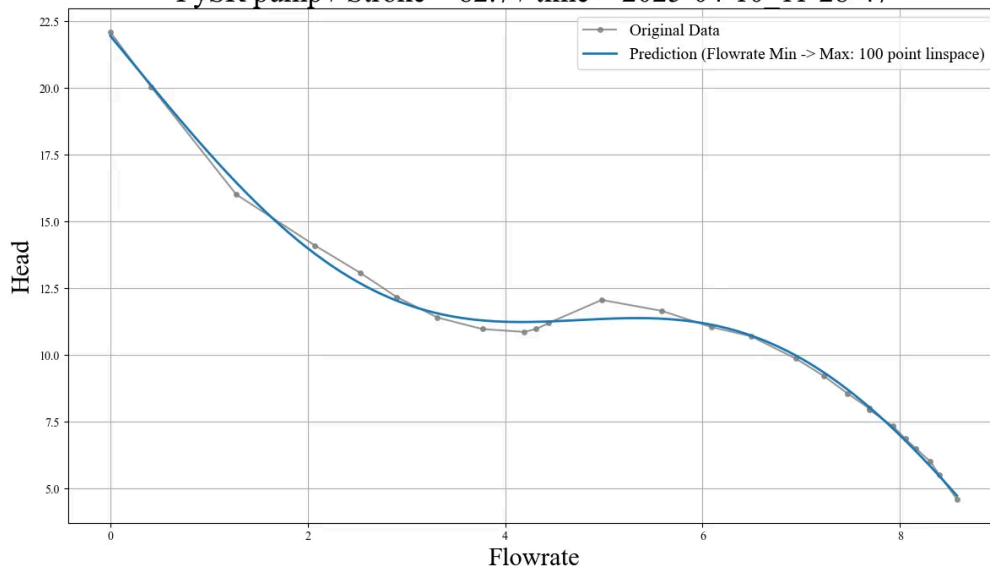
Stroke_value: 74.8 | RMSE: 0.25090 | r2: 0.99620 | Time: 1148.57981sec
Best equation: $(x_0 - 1.4932384 \cdot \cos(0.7094332 \cdot (x_0 + 1.9455647))) \cdot (-2.046656) - 1 \cdot (-20.735489)$

✓ 최종 정리된 수식:

$$f(x) = -2.046656x + 3.055841 \cdot \cos(0.7094332x + 1.380706) + 20.735489$$

Data 5: stroke=82.7

PySR pump / Stroke = 82.7 / time = 2025-04-10_11-28-47



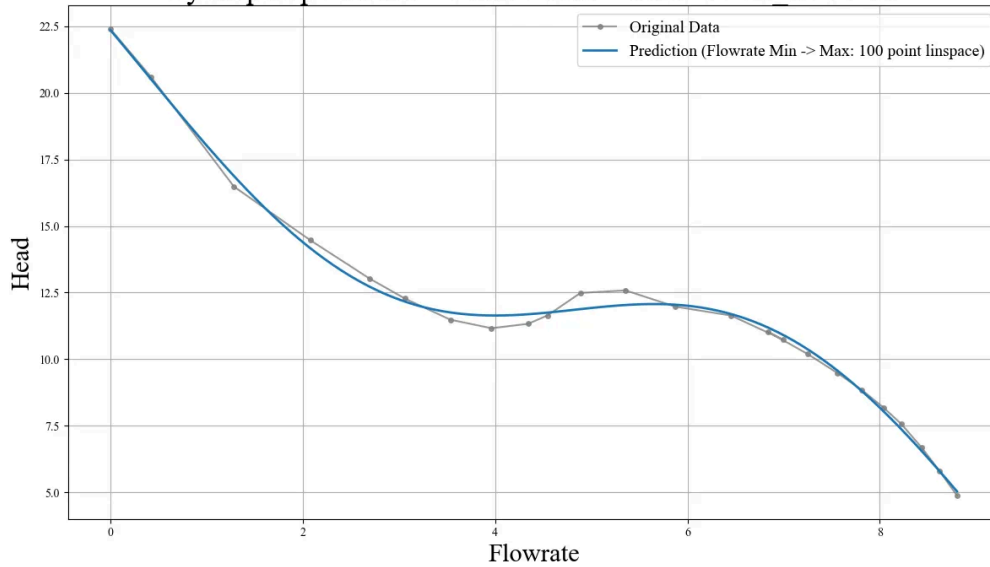
Stroke_value: 82.7 | RMSE: 0.24653 | r2: 0.99625 | Time: 1075.73757sec
Best equation: $21.526495 + (x_0 - 1.5800593 \cdot \cos(x_0 \cdot (-0.68809354) - 1 \cdot 1.4442109)) \cdot (-2.1507673)$

✓ 최종 전개된 수식:

$$f(x) = -2.1507673x + 3.397003 \cdot \cos(-0.68809354x - 1.4442109) + 21.526495$$

Data 6: stroke=93.95

PySR pump / Stroke = 93.95 / time = 2025-04-10 11-50-00

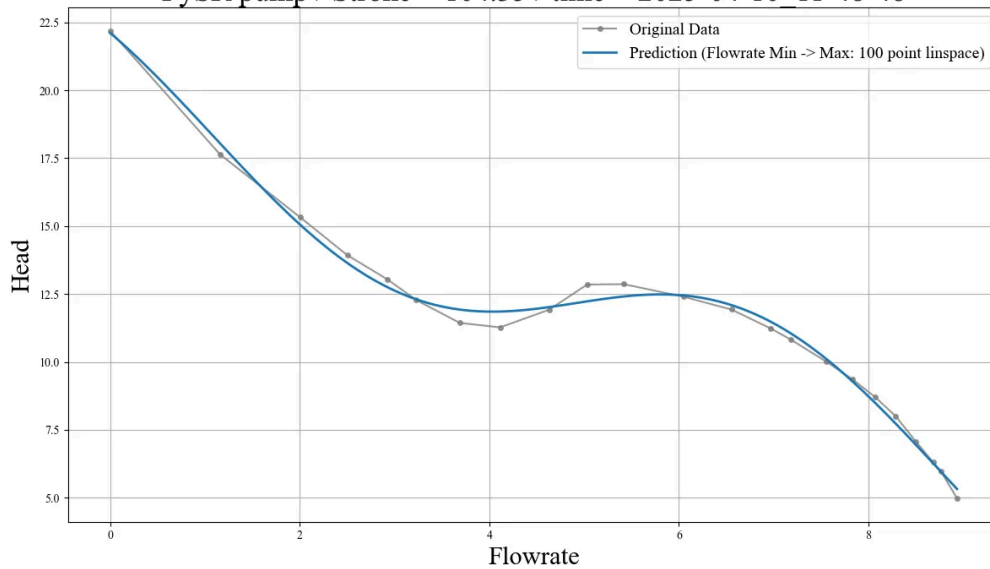


✅ 전개된 최종 수식:

$$f(x) = -2.0008087x + 3.4007766 \cdot \cos(0.70412683x + 1.316023) + 21.505032$$

🔍 Data 7: stroke=104.35

PySR pump / Stroke = 104.35 / time = 2025-04-10 11-48-48



✅ 정리된 수식 (표준 형태):

$$f(x) = -1.6076496x + 2.6261282 \cdot \cos(-0.81846154x - 0.6879015) + 20.074152$$

🔍 Data 1: stroke=15.85

🔍 Data 2: stroke=45.8

🔍 Data 3: stroke=69.3

🔍 Data 4: stroke=74.8

🔍 Data 5: stroke=82.7

🔍 Data 6: stroke=93.95

🔍 Data 7: stroke=104.35

📊 7개 실험 결과 요약

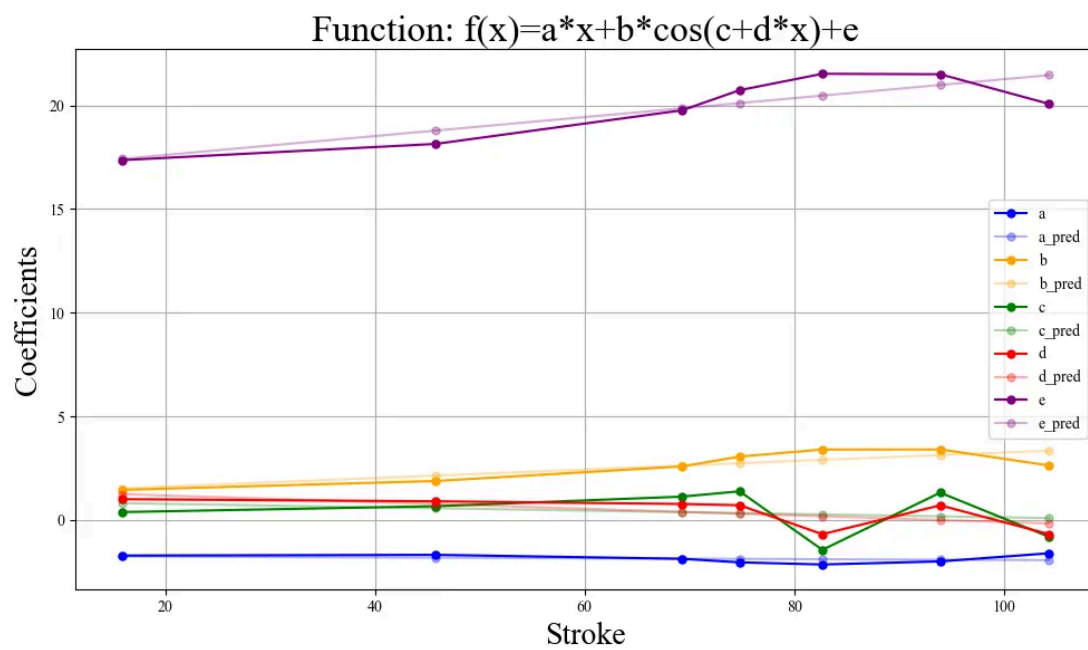
$$f(x) = a \cdot x + b \cdot \cos(c + d \cdot x) + e$$

실험	a	b	c	d	e	RMSE	r2-score
Data 1: stroke=15.85	-1.7189162	1.450246	0.3792143	1	17.361654	0.16761	0.99822
Data 2: stroke=45.8	-1.684188	1.879294	0.66195387	0.8985187	18.144154	0.18892	0.99761
Data 3: stroke=69.3	-1.875963	2.581567	1.1242205	0.7695822	19.765572	0.22449	0.99621
Data 4: stroke=74.8	-2.046656	3.055841	1.380706	0.7094332	20.735489	0.25090	0.99620
Data 5: stroke=82.7	-2.1507673	3.397003	-1.4442109	-0.68809354	21.526495	0.24653	0.99625
Data 6: stroke=93.95	-2.0008087	3.4007766	1.316023	0.70412683	21.505032	0.26476	0.99558
Data 7: stroke=104.35	-1.6076496	2.6261282	-0.81846154	-0.6879015	20.074152	0.29490	0.99388

🔍 원본 Stroke를 넣고 수식의 a, b, c, d, e(coefficients) 예측

예) X_train → Stroke, y_train → a / X_test → Stroke, y_test → pred_a

🔍 LinearRegression 적용 해보기(1)

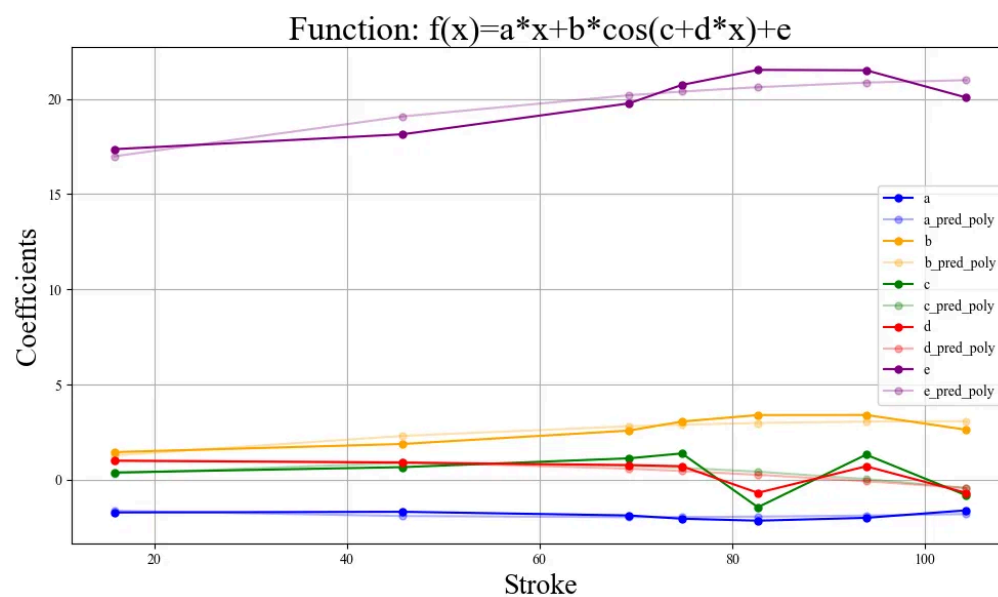


Parameter	RMSE	MAE	R2
a vs pred_a	0.1814	0.1456	0.0909
b vs pred_b	0.3806	0.3116	0.6947
c vs pred_c	0.9937	0.8697	0.0498
d vs pred_d	0.5214	0.4648	0.4232
e vs pred_e	0.7671	0.6261	0.7341

LinearRegression 말고 다른 모델 적용 해보기(2) → LinearRegressor+Polynomial

하는 이유: LinearRegression이 비선형성을 잘 해석 하지 못하는 것으로 보임

LinearRegression + PolynomialFeatures(degree=2)



Parameter	RMSE	MAE	R2
a vs pred_poly_a	0.1554	0.1442	0.3332
b vs pred_poly_b	0.3344	0.3144	0.7644
c vs pred_poly_c	0.9299	0.7067	0.1680
d vs pred_poly_d	0.4881	0.3504	0.4945
e vs pred_poly_e	0.6964	0.6506	0.7808

📌 중간 결과

(1), (2) 실험은 차이 없음

TODO(PC24-11, 2025-04-10 15:46):

🔍 LinearRegression+Poly(2) 말고 다른 모델 적용 해보기(3) → Polynomial + SVR

왜 조합하냐? → 데이터가 작기 때문에 polynomial로 데이터를 늘려서 SVR을 시도!

—> 이것은 수식을 조금 더 근사화 시키고 위의 과정 진행 해야 될 것으로 판단 됨

📌 금일 결론 및 결과 분석

maxsize(수식길이)를 늘려야 될 것으로 판단이 됨

